

Chapter 17

Introduction to Spring MVC

In Chapter 16, “Model 2 and the MVC Pattern” you learned that the widely used design of modern web applications follow the MVC pattern. You also learned the advantages of the Model 2 architecture and how to build Model 2 applications. Spring MVC is a framework that helps developers write MVC applications more rapidly.

This chapter starts by discussing the benefits of Spring MVC and how it expedites Model 2 application development. It also discusses some basic components of Spring MVC, such as the Dispatcher Servlet, and teaches you how to write “old-style” controllers that were the only way of writing controllers in older versions of Spring prior to version 2.5. Another type of controller is covered in Chapter 18, “Annotation-based Controllers.” The old style controller is discussed here because you might still have to work with legacy code written with older versions of Spring. For new developments, however, you should use annotation-based controllers.

Introducing Spring MVC configuration is another objective of this chapter. Most Spring MVC applications will have an XML file for declaring various beans that are used in the applications.

The Benefits of Spring MVC

When writing a Model 2 application without a framework, it is your responsibility to write a dispatcher servlet and controller classes. Your dispatcher servlet must be capable of doing these things:

1. Determine from the URI what action to invoke.
2. Instantiate the correct controller class.
3. Populate a form bean with request parameter values.
4. Call the correct method in the controller object.
5. Forward control to a view (JSP page).

Spring MVC is an MVC framework that employs a dispatcher servlet that invokes methods in controllers and forwards control to a view. This is the first benefit of using Spring MVC: You don’t need to write your own dispatcher servlet. Here is the list of features that Spring MVC is equipped with to make development more rapid.

- Spring MVC provides a dispatcher servlet, saving your writing one.
- Spring MVC employs an XML-based configuration file that you can edit without recompiling the application.
- Spring MVC instantiates controller classes and populates beans with user inputs.
- Spring MVC automatically binds user input with the correct type. For example, Spring MVC can automatically parse a string and sets a property of type float or

- decimal.
- Spring MVC validates user input and redirects the user back to the input form if validation failed. Input validation is optional and can be done programmatically or declaratively. On top of that, Spring MVC provides built-in validators for most of the tasks you may encounter when building a web application.
- Spring MVC is part of the Spring framework. You get everything Spring has to offer.
- Spring MVC supports internationalization and localization. This means, you can display messages in multiple languages depending on the user locale.
- Spring MVC supports multiple view technologies. Most of the time you'll be using JSP, but other technologies are supported, including Velocity and FreeMarker.

Spring MVC DispatcherServlet

Recall that in Chapter 16, “Model 2 and the MVC Pattern” you built a simple MVC framework that consisted of a servlet that acted as a dispatcher. With Spring MVC, you don't have to do that. Spring MVC comes with a dispatcher servlet that you can instantly use. Its fully qualified name is **org.springframework.web.servlet.DispatcherServlet**.

To use this servlet, you need to configure it in your deployment descriptor (**web.xml** file) using the **servlet** and **servlet-mapping** elements, like this.

```
<servlet>
  <servlet-name>springmvc</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
  <servlet-name>springmvc</servlet-name>
  <!-- map all requests to the DispatcherServlet -->
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

The **load-on-startup** element under **<servlet>** is optional. If it is present, it will load the servlet and call its **init** method when the application is started. Without the **load-on-startup** element, the servlet will be loaded when it is first requested.

By itself the dispatcher servlet will use many default components that come with Spring MVC. In addition, at initialization it will look for a configuration file in the **WEB-INF** directory of the application. The name of the XML file must conform to this pattern

```
servletName-servlet.xml
```

where *servletName* is the name given to the Spring dispatcher servlet in the deployment descriptor. If you have given the servlet the name **springmvc**, you will need to have a **springmvc-servlet.xml** file under the **WEB-INF** directory of your application directory.

However, you can place your Spring MVC configuration file anywhere within your application directory as long as you tell the dispatcher servlet where to find it. You do this by using an **init-param** element under the servlet declaration. The **init-param** element

would have a **param-name** element that has the value **contextConfigLocation**. It would also have a **param-value** element containing the path to your configuration file. For example, you can change the default name and location of the configuration file to **/WEB-INF/config/simple-config.xml** by using this **init-param** element.

```
<servlet>
  <servlet-name>springmvc</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/config/simple-config.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

The Controller Interface

Prior to Spring 2.5, the only way to write a controller was by implementing the **org.springframework.web.servlet.mvc.Controller** interface. This interface exposes a **handleRequest** method that must be overridden by implementing classes. Here is the signature of the method.

```
ModelAndView handleRequest(HttpServletRequest request,
                          HttpServletResponse response)
```

An implementation has access to the **HttpServletRequest** and **HttpServletResponse** of the corresponding request. The implementation must also return a **ModelAndView** that contains a view path or a view path and a model.

A controller implementing the **Controller** interface can only handle one single action. On the other hand, an annotation-based controller can house many request-handling methods and do not have to implement any interface. You will learn about the latter in Chapter 18, “Annotation-based Controllers.”

Your First Spring MVC Application

The **app17a** sample application showcases a basic Spring MVC application. The application is very similar to the **app16b** application you learned in Chapter 16. It was deliberately made so to show you how Spring MVC works. The **app17a** application also has two controllers that are similar to the ones in **app17b**.

The Directory Structure

Figure 17.1 shows the directory structure of **app17a**. Note that the **WEB-INF/lib** directory contains all the jar files required by Spring MVC. Of special interest is the **spring-webmvc-x.y.z.jar** file, which contains the **DispatcherServlet** class. Also note that

Spring MVC depends on the Apache Commons Logging component and without it your Spring MVC applications won't work. You can download this component from this site.

<http://commons.apache.org/proper/commons-logging/>

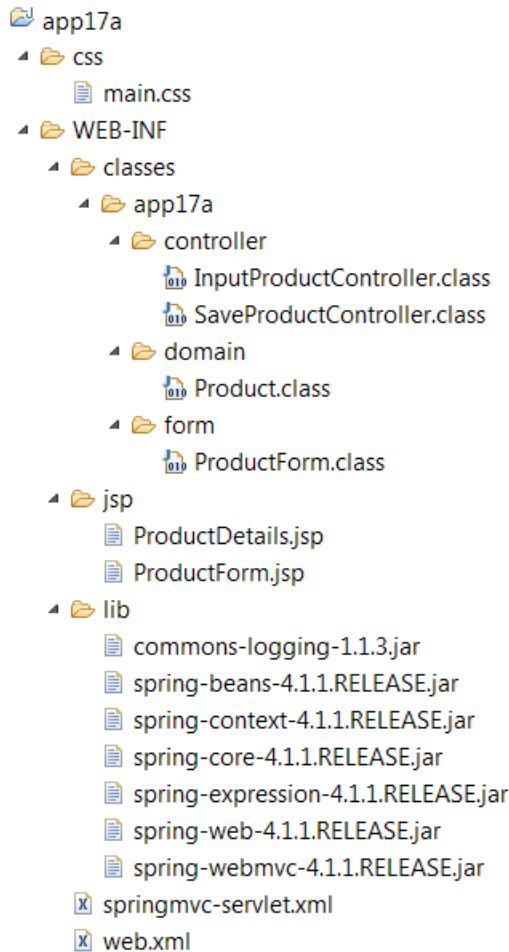


Figure 17.1: The directory structure of app17a

The JSP pages for this application are stored under `/WEB-INF/jsp` to keep them from direct access.

The Deployment Descriptor and Spring MVC Configuration File

Now look at the deployment descriptor (`web.xml` file) in Listing 17.1

Listing 17.1: The deployment descriptor for app17a

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0"
  xmlns="http://java.sun.com/xml/ns/javaee"
```

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
➡ http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">

<servlet>
  <servlet-name>springmvc</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
  <servlet-name>springmvc</servlet-name>
  <!-- map all requests to the DispatcherServlet -->
  <url-pattern>/</url-pattern>
</servlet-mapping>

</web-app>

```

Nothing spectacular here. You're just telling the servlet/JSP container that you want to use the Spring MVC dispatcher servlet and map it to all URLs by using `/` in the **url-pattern** element. Since there is no **init-param** element under the **servlet** element, the Spring MVC configuration file is assumed to be under **/WEB-INF** and follow the usual naming convention.

Next, examine the Spring MVC configuration file (**springmvc-servlet.xml** file) in Listing 17.2.

Listing 17.2: The Spring MVC configuration file

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean name="/product_input.action"
    class="app17a.controller.InputProductController"/>
  <bean name="/product_save.action"
    class="app17a.controller.SaveProductController"/>

</beans>

```

Here you declare two controller classes, **InputProductController** and **SaveProductController**, and map them to **/product_input.action** and **/product_save.action**, respectively. The controllers are discussed in the next section.

The Controllers

The **app17a** application has two “old-style” controllers, **InputProductController** and **SaveProductController**. Both implement the **Controller** interface. The **InputProductController** class is given in Listing 17.3 and **SaveProductController** in Listing 17.4.

Listing 17.3: The InputProductController class

```
package appl7a.controller;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;

public class InputProductController implements Controller {

    private static final Log logger = LogFactory
        .getLog(InputProductController.class);

    @Override
    public ModelAndView handleRequest(HttpServletRequest request,
        HttpServletResponse response) throws Exception {
        logger.info("InputProductController called");
        return new ModelAndView("/WEB-INF/jsp/ProductForm.jsp");
    }
}
```

The **handleRequest** method of the **InputProductController** class simply returns a **ModelAndView** that contains a view with no model. In this case, the request will be forwarded to the **/WEB-INF/jsp/ProductForm.jsp** page.

Listing 17.4: The SaveProductController class

```
package appl7a.controller;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;
import appl7a.domain.Product;
import appl7a.form.ProductForm;

public class SaveProductController implements Controller {

    private static final Log logger = LogFactory
        .getLog(SaveProductController.class);

    @Override
    public ModelAndView handleRequest(HttpServletRequest request,
        HttpServletResponse response) throws Exception {
        logger.info("SaveProductController called");
        ProductForm productForm = new ProductForm();
        // populate action properties
        productForm.setName(request.getParameter("name"));
        productForm.setDescription(request.getParameter(
            "description"));
    }
}
```

```

        productForm.setPrice(request.getParameter("price"));

        // create model
        Product product = new Product();
        product.setName(productForm.getName());
        product.setDescription(productForm.getDescription());
        try {
            product.setPrice(
                Float.parseFloat(productForm.getPrice()));
        } catch (NumberFormatException e) {
        }

        // insert code to save Product

        return new ModelAndView("/WEB-INF/jsp/ProductDetails.jsp",
            "product", product);
    }
}

```

The **handleRequest** method in the **SaveProductController** class creates a **ProductForm** object and populates it using the request parameters. It then creates a **Product** object that gets its property values from the **ProductForm**. Since the **price** property of the **ProductForm** is a **String** and its counterpart in the **Product** class is a float, some parsing is necessary. In the next chapters you will learn how Spring MVC eliminates the need for form beans like **ProductForm** and makes things much simpler.

The **handleRequest** method in **SaveProductController** concludes by returning a **ModelAndView** that contains a view path, a model name, and a model (the **Product** object). The model added to the **ModelAndView** object will be available to the target view for display.

The View

The **app17a** application comes with two JSP pages, the **ProductForm.jsp** page (given in Listing 17.5) and the **ProductDetails.jsp** page (printed in Listing 17.6).

Listing 17.5: The ProductForm.jsp page

```

<!DOCTYPE html>
<html>
<head>
<title>Add Product Form</title>
<style type="text/css">@import url(css/main.css);</style>
</head>
<body>

<div id="global">
<form action="product_save.action" method="post">
    <fieldset>
        <legend>Add a product</legend>
        <label for="name">Product Name: </label>
        <input type="text" id="name" name="name" value=""
            tabindex="1">
        <label for="description">Description: </label>
    
```

```

        <input type="text" id="description" name="description"
            tabindex="2">
        <label for="price">Price: </label>
        <input type="text" id="price" name="price" tabindex="3">
        <div id="buttons">
            <label for="dummy"> </label>
            <input id="reset" type="reset" tabindex="4">
            <input id="submit" type="submit" tabindex="5"
                value="Add Product">
        </div>
    </fieldset>
</form>
</div>
</body>
</html>

```

This is not a place to discuss HTML and CSS, but we'd like to stress here that the HTML in Listing 17.5 has been written with proper design in mind. Among others, we were not using `<table>` to lay out the input fields.

Listing 17.6: The `ProductDetails.jsp` page

```

<!DOCTYPE html>
<html>
<head>
<title>Save Product</title>
<style type="text/css">@import url(css/main.css);</style>
</head>
<body>
<div id="global">
    <h4>The product has been saved.</h4>
    <p>
        <h5>Details:</h5>
        Product Name: ${product.name}<br/>
        Description: ${product.description}<br/>
        Price: $$${product.price}
    </p>
</div>
</body>
</html>

```

The **`ProductDetails.jsp`** page has access to the **`Product`** object passed by the **`SaveProductController`** with the model attribute name **`product`**. We used the JSP Expression Language expressions to display various properties of the **`Product`** object.

Testing the Application

To test the application, direct your browser to this URL:

`http://localhost:8080/app17a/product_input.action`

You will see the familiar Product form like the one in Figure 17.2. Type in values in the empty fields and click the Add Product button. You will see the product properties are shown in the next page.

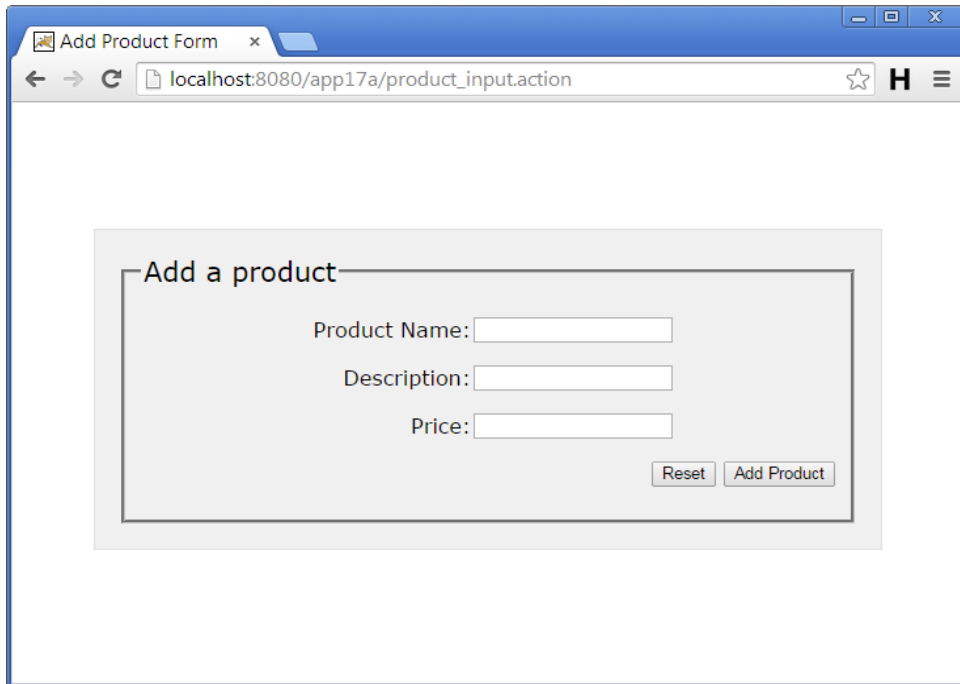


Figure 17.2: The Product form in app17a

The View Resolver

The view resolver in Spring MVC is responsible for resolving views. To use and configure the view resolver, declare a **viewResolver** bean in your configuration file, such as this one.

```
<bean id="viewResolver" class="org.springframework.web.servlet.  
view.InternalResourceViewResolver">  
  <property name="prefix" value="/WEB-INF/jsp/" />  
  <property name="suffix" value=".jsp" />  
</bean>
```

The **viewResolver** bean above configures two properties, **prefix** and **suffix**. As a result, your view paths will be shorter. Instead of setting the view path to **/WEB-INF/jsp/myPage.jsp**, for example, you just write **myPage** and the view resolver will prefix and suffix the string.

As an example, consider the **app17b** application, which is similar to **app17a**. However, the name and the location of the configuration file have been changed. In addition, it configures the default view resolver to add a prefix and a suffix to all view paths.

Figure 17.3 shows the directory structure of **app17b**.

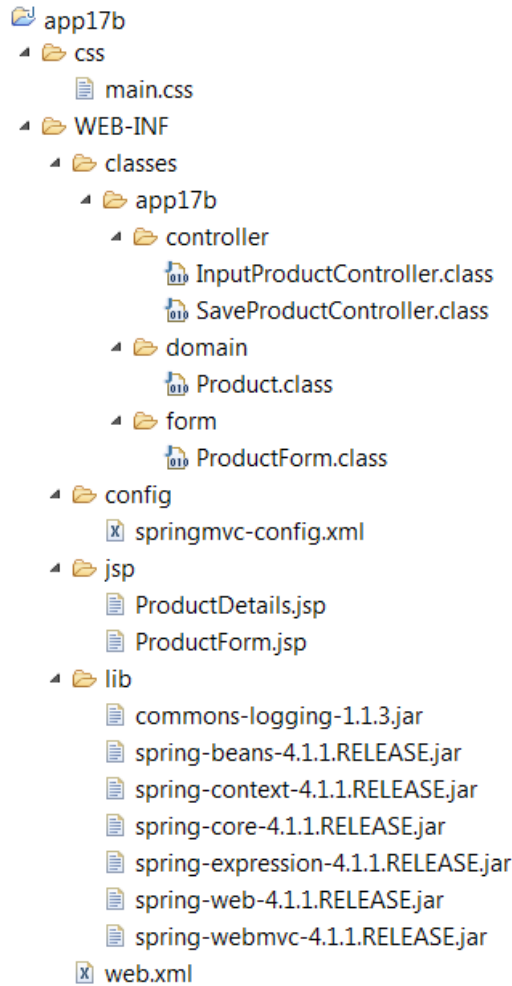


Figure 17.3: The directory structure of app17b

In **app17b** the Spring MVC configuration file has been renamed **springmvc-config.xml** and moved to **/WEB-INF/config**. In order to tell Spring MVC where to find it, you need to pass the location of the file to the Spring MVC dispatcher servlet. Listing 17.7 shows the deployment descriptor (**web.xml** file) for **app17b**.

Listing 17.7: The deployment descriptor for app17b

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0"
  xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">

  <servlet>
    <servlet-name>springmvc</servlet-name>
    <servlet-class>
```

```

        org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>
            /WEB-INF/config/springmvc-config.xml
        </param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
    <servlet-name>springmvc</servlet-name>
    <url-pattern>*.action</url-pattern>
</servlet-mapping>
</web-app>

```

Pay special attention to the **init-param** element in the **web.xml** file. To refer to a configuration file that is not using the default naming and location, you need to use the **contextConfigLocation** initial parameter. Its value should be the path to the configuration file, relative to the application directory.

The configuration file for **app17b** is given in Listing 17.8.

Listing 17.8: The Spring MVC configuration file for app17b

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean name="/product_input.action"
        class="app17b.controller.InputProductController"/>
    <bean name="/product_save.action"
        class="app17b.controller.SaveProductController"/>
    <bean id="viewResolver"
        class="org.springframework.web.servlet.view.
➡ InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/jsp/" />
        <property name="suffix" value=".jsp" />
    </bean>
</beans>

```

To test the application, direct your browser to this URL:

`http://localhost:8080/app17b/product_input.action`

You will see a form similar to Figure 17.2.

Summary

This chapter is a gentle introduction to Spring MVC. It teaches you how to write simple applications similar to the examples in Chapter 16. You don't need to write your own dispatcher servlet in Spring MVC and controllers can be written by implementing the **Controller** interface. This is the old style controller. In Spring 2.5 and later, there is a better way of writing controllers, i.e. by using annotations. Chapter 18, "Annotation-Based Controllers" discusses this type of controller.